

## Section 1: AI/ML/DL Fundamentals

**Q: What is the difference between AI, ML, and Deep Learning?**

A: AI is the broadest concept of machines mimicking human intelligence. ML is a subset of AI where systems learn from data. Deep Learning is a subset of ML using neural networks with multiple layers.

**Q: What are the three main types of Machine Learning?**

A: Supervised Learning (labeled data), Unsupervised Learning (unlabeled data), and Reinforcement Learning (reward-based learning).

**Q: What is the key difference between supervised and unsupervised learning?**

A: Supervised learning uses labeled training data with known outputs, while unsupervised learning finds patterns in unlabeled data without predefined outputs.

**Q: What is a neural network?**

A: A computational model inspired by biological neurons, consisting of interconnected nodes (neurons) organized in layers that process and transform input data.

**Q: What makes Deep Learning "deep"?**

A: The presence of multiple hidden layers between input and output layers, allowing the network to learn hierarchical representations of data.

## Section 2: Generative AI Fundamentals

**Q: What is Generative AI?**

A: AI systems that can create new content (text, images, code, audio) based on patterns learned from training data, rather than just classifying or predicting.

**Q: What are common use cases for Generative AI?**

A: Content creation, code generation, chatbots, image synthesis, text summarization, translation, data augmentation, and creative design.

**Q: What are the main types of Generative AI models?**

A: Large Language Models (LLMs), Diffusion Models, GANs (Generative Adversarial Networks), VAEs (Variational Autoencoders), and Transformer-based models.

**Q: What is a transformer architecture?**

A: A neural network architecture using self-attention mechanisms to process sequential data in parallel, forming the foundation of modern LLMs.

**Q: What is the attention mechanism in transformers?**

A: A technique that allows the model to focus on relevant parts of input when generating output, weighing the importance of different tokens.

## Section 3: Large Language Models (LLMs)

**Q: What is a Large Language Model?**

A: A neural network trained on massive text datasets to understand and generate human-like text, typically with billions of parameters.

**Q: What is a token in LLM context?**

A: A unit of text (word, subword, or character) that the model processes. Text is broken into tokens before processing.

**Q: What is the context window in LLMs?**

A: The maximum number of tokens an LLM can process at once, determining how much information it can consider when generating responses.

**Q: What's the difference between open source and closed source LLMs?**

A: Open source models have publicly available weights and code (Llama, Mistral), while closed source models are proprietary with API-only access (GPT-4, Claude).

**Q: What are model parameters?**

A: Learnable weights in the neural network that determine the model's behavior, adjusted during training to improve performance.

**Q: What is temperature in LLM generation?**

A: A parameter controlling randomness in output. Lower values (0.0-0.3) produce deterministic responses, higher values (0.7-1.0) increase creativity and variety.

**Q: What is top-p (nucleus) sampling?**

A: A generation strategy that samples from the smallest set of tokens whose cumulative probability exceeds threshold p, balancing quality and diversity.

**Q: What is a tokenizer in LLMs?**

A: A component that converts raw text into tokens and vice versa, breaking down text into manageable units using algorithms like Byte-Pair Encoding (BPE), WordPiece, or SentencePiece that the model can process.

**Q: What are hyperparameters in LLMs?**

A: Configuration settings that control the training and inference process, including learning rate,

batch size, number of layers, hidden dimensions, attention heads, temperature, top-p, top-k, max tokens, and presence/frequency penalties.

**Q: What is the difference between parameters and hyperparameters?**

A: Parameters are internal model weights learned during training (e.g., 7B, 13B, 70B parameters), while hyperparameters are external settings configured before training or inference that control how the model learns or generates output.

**Q: What is multimodality in LLMs?**

A: The ability of a model to process and generate multiple types of data (modalities) such as text, images, audio, and video, enabling richer understanding and more versatile applications beyond text-only interactions.

**Q: What is fine-tuning in LLMs?**

A: The process of taking a pre-trained model and further training it on a specific dataset to adapt it for particular tasks, domains, or behaviors while preserving its general language understanding.

**Q: What is the difference between pre-training and fine-tuning?**

A: Pre-training involves training a model from scratch on massive general datasets to learn language patterns, while fine-tuning adapts a pre-trained model to specific tasks using smaller, targeted datasets.

**Q: What is few-shot learning in LLMs?**

A: The ability of LLMs to learn and perform new tasks from just a few examples provided in the prompt, without requiring additional training or fine-tuning.

**Q: What is zero-shot learning?**

A: The capability of an LLM to perform tasks it has never explicitly been trained on, using only its general language understanding and task instructions without any examples.

**Q: What are some common training data sources for LLMs?**

A: Web pages, books, Wikipedia, scientific papers, GitHub code repositories, Common Crawl, news articles, online forums like Reddit, and research databases.

**Q: What is prompt engineering?**

A: The practice of carefully designing and structuring input prompts to guide LLMs toward producing desired outputs, including techniques like instructions, examples, and context framing.

**Q: What is hallucination in LLMs?**

A: When a model generates false, nonsensical, or fabricated information that sounds plausible but is not grounded in its training data or provided context.

## Section 4: Specific Models

**Q: What are the main OpenAI models?**

A: GPT-4 (most capable, multimodal), GPT-4 Turbo (faster, cheaper), GPT-3.5 Turbo (fast, cost-effective for simple tasks).

**Q: What is Llama?**

A: Meta's open-source LLM family, available in various sizes (7B, 13B, 70B parameters) for diverse computational requirements.

**Q: What is Phi-3?**

A: Microsoft's small language model (3.8B parameters) optimized for efficiency, capable of running on edge devices with strong reasoning abilities.

**Q: What is Ollama?**

A: A tool for running LLMs locally on your machine, supporting models like Llama, Mistral, and others with simple command-line interface.

**Q: What is LM Studio?**

A: A desktop application for discovering, downloading, and running local LLMs with a user-friendly interface and model management features.

**Q: What are the Phi-3 model variants?**

A: Phi-3-mini (3.8B parameters), Phi-3-small (7B parameters), Phi-3-medium (14B parameters), and Phi-3.5 variants including MoE and Vision models.

**Q: What are common Ollama commands?**

A: Essential commands include:

- `ollama pull <model-name>` - Download a model (e.g., `ollama pull llama2`, `ollama pull phi3`)
- `ollama run <model-name>` - Run and interact with a model
- `ollama list` - Show all downloaded models
- `ollama rm <model-name>` - Remove a model
- `ollama serve` - Start the Ollama server
- `ollama ps` - Show running models
- `ollama stop <model-name>` - Stop a running model
- `ollama show <model-name>` - Display model information
- `ollama cp <source> <destination>` - Copy a model
- `ollama create <model-name> -f Modelfile` - Create a custom model from a Modelfile

## Section 5: Basic Prompt Engineering

**Q: What is a prompt?**

A: The input text or instruction given to an LLM to guide its response generation.

**Q: What is a system prompt?**

A: Instructions that set the AI's behavior, role, and constraints for the entire conversation, typically invisible to end users.

**Q: What is a user prompt?**

A: The specific query or instruction from the user that the AI responds to directly.

**Q: What are the key elements of effective prompts?**

A: Clear instructions, relevant context, specific output format requirements, examples when needed, and constraints or limitations.

**Q: What is a prompt template?**

A: A reusable prompt structure with placeholders for variables usually within curly braces {} or angle brackets <>, allowing consistent formatting across multiple queries.

**Q: What is few-shot prompting?**

A: Providing the model with examples of desired input-output pairs before asking it to perform a task.

**Q: What is zero-shot prompting?**

A: Asking the model to perform a task without providing any examples, relying solely on its pre-trained knowledge.

## Section 6: Advanced Prompt Engineering

**Q: What is Chain-of-Thought (CoT) prompting?**

A: A technique that encourages the model to show its reasoning process step-by-step before providing the final answer.

**Q: What is Tree-of-Thought (ToT) prompting?**

A: An approach that explores multiple reasoning paths simultaneously, evaluating different branches before selecting the best solution.

**Q: What is ReAct prompting?**

A: A framework combining reasoning and acting, where the model alternates between thinking steps and taking actions to solve problems.

**Q: When should you use Chain-of-Thought?**

A: For complex reasoning tasks, mathematical problems, multi-step logic, and when interpretability of the reasoning process is important.

**Q: What is the key difference between CoT and ToT?**

A: CoT follows a single linear reasoning path, while ToT explores multiple parallel paths and backtracks if needed, like a search algorithm.

**Q: What does ReAct stand for?**

A: Reasoning + Acting, representing the interleaved pattern of thought and action steps.

## Section 7: No-Code/Low-Code Platforms

**Q: What is Microsoft Copilot Studio?**

A: A low-code platform for building AI-powered chatbots and agents using natural language, with built-in connectors and templates.

**Q: What is Google Agent Space/ADK?**

A: Google's platform for creating and deploying AI agents with visual workflows and pre-built components for common tasks.

**Q: What are the benefits of no-code AI platforms?**

A: Faster development, lower technical barrier, rapid prototyping, pre-built integrations, and easier maintenance for non-developers.

**Q: When should you use no-code vs code-based approaches?**

A: No-code for simple workflows and rapid prototypes; code-based for complex logic, custom integrations, and production-scale applications.

## Section 8: Development Environment

**Q: What is Git?**

A: A distributed version control system for tracking changes in code, enabling collaboration and maintaining project history.

**Q: What is the difference between Git and GitHub?**

A: Git is the version control system; GitHub is a cloud platform for hosting Git repositories with collaboration features.

**Q: What are the essential Git commands?**

A: git init, git clone, git add, git commit, git push, git pull, git branch, git merge, git status.

**Q: What is GitHub Copilot?**

A: An AI-powered code completion tool that suggests code snippets and entire functions based on context and comments.

**Q: What is Cursor?**

A: An AI-first code editor built on VS Code, with native AI features for code generation, editing, and debugging.

**Q: What makes AI code editors different from traditional IDEs?**

A: Built-in LLM integration, contextual code suggestions, natural language commands, automated refactoring, and intelligent debugging assistance.

## Section 9: LangChain Framework

**Q: What is LangChain?**

A: A framework for developing applications powered by language models, providing modular components for building LLM-based apps.

**Q: What are the core LangChain components?**

A: Models (LLMs), Prompts (templates), Parsers (output formatting), Chains (sequences), Agents (decision-makers), Memory (context retention).

**Q: What is a LangChain chain?**

A: A sequence of components that process data in steps, where the output of one component feeds into the next.

**Q: What is a LangChain agent?**

A: An autonomous component that uses LLMs to decide which actions to take and in what order to achieve a goal.

**Q: What types of memory does LangChain support?**

A: ConversationBufferMemory, ConversationSummaryMemory, ConversationBufferWindowMemory, and VectorStoreMemory.

**Q: What are LangChain tools?**

A: Functions or APIs that agents can call to interact with external systems (search, calculators, databases, APIs).

**Q: What is an output parser in LangChain?**

A: A component that structures LLM outputs into specific formats (JSON, lists, custom objects) for easier programmatic use.

**Q: How do you debug LangChain applications?**

A: Use verbose mode, enable callbacks, inspect intermediate outputs, use LangSmith tracing, and implement error handling.

## Section 10: Agentic AI Principles

**Q: What is an AI agent?**

A: An autonomous system that perceives its environment, makes decisions, and takes actions to achieve specific goals.

**Q: What are the key characteristics of AI agents?**

A: Autonomy (independent operation), reactivity (responds to environment), proactivity (goal-directed), and social ability (multi-agent interaction).

**Q: What is perception in agentic AI?**

A: The agent's ability to gather and interpret information from its environment through sensors, APIs, or data sources.

**Q: What is decision-making in agents?**

A: The process of selecting actions based on current state, goals, and available options using reasoning or learning.

**Q: What is agent memory?**

A: The ability to store and retrieve information about past interactions, decisions, and learned patterns to improve future performance.

**Q: What is the perception-action cycle?**

A: The continuous loop where agents perceive the environment, process information, make decisions, and execute actions.

## Section 11: Agentic AI Frameworks

**Q: What is AutoGen?**

A: Microsoft's framework for building multi-agent conversational systems where agents collaborate to solve complex tasks.

**Q: What is a conversable agent in AutoGen?**

A: An agent that can send and receive messages, enabling communication with other agents and humans.

**Q: What are the types of agents in AutoGen?**

A: AssistantAgent (LLM-powered), UserProxyAgent (executes code/tools), GroupChat (manages multiple agents), and custom agents.

**Q: What is a multi-agent system?**

A: Multiple AI agents working together, each with specialized roles, to solve complex problems through collaboration.

**Q: What is a sequential agent pattern?**

A: Agents execute tasks one after another in a predefined order, with each agent's output feeding the next.

**Q: What is a supervisor/hierarchy pattern?**

A: A manager agent coordinates and delegates tasks to worker agents, monitoring progress and making high-level decisions.

**Q: What is agent handoff?**

A: The process of one agent transferring control or information to another agent to continue task execution.

## **Section 12: Model Context Protocol (MCP)**

**Q: What is Model Context Protocol?**

A: A standardized protocol for AI agents to communicate with tools, data sources, and other agents with consistent context sharing. It was introduced by Anthropic and adopted by many.

**Q: Why is MCP important?**

A: It provides interoperability between different AI systems, standardizes tool integration, and ensures consistent context propagation.

**Q: What problems does MCP solve?**

A: Fragmented tool integrations, inconsistent context handling, lack of interoperability between agent frameworks, and complex multi-agent communication.

**Q: What are MCP servers?**

A: Services that expose tools and data sources to agents through the MCP protocol, handling requests and returning results.

**Q: What are MCP clients?**

A: Agents or applications that consume MCP servers, making requests for tools and data through the standard protocol.

**Q: How does MCP benefit multi-agent systems?**

A: Enables seamless communication, shared context understanding, standardized tool access, and easier integration of new capabilities.

**Q: What are 3 ways to connect to MCP servers?**

A: There are three primary connection methods for MCP servers:

1. **STDIO (Standard Input/Output)**: Uses standard input/output streams for direct process communication between local processes on the same machine, providing optimal performance with no network overhead. This method is mainly used for local integrations where the server runs in the same environment as the client.

2. **HTTP + SSE (Server-Sent Events)**: Used for remote connections, with HTTP handling client requests and Server-Sent Events managing server responses and streaming.

3. **Streamable HTTP transport:** Uses HTTP POST for client-to-server messages with optional Server-Sent Events for streaming capabilities. This transport enables remote server communication and supports standard HTTP authentication methods including bearer tokens, API keys, and custom headers. MCP recommends using OAuth to obtain authentication tokens.

## Section 13: Multi-Agent Design

**Q: What is agent orchestration?**

A: Coordinating multiple agents' activities to achieve a common goal, managing dependencies, and handling failures.

**Q: What is consensus in multi-agent systems?**

A: The process where multiple agents agree on a decision or action through voting, negotiation, or other mechanisms.

**Q: What is agent specialization?**

A: Designing agents with specific expertise or capabilities to handle particular types of tasks efficiently.

**Q: What are common multi-agent communication patterns?**

A: Broadcast (one-to-all), request-reply (one-to-one), publish-subscribe (event-driven), and consensus protocols.

**Q: How do you handle conflicts in multi-agent systems?**

A: Priority-based resolution, voting mechanisms, supervisor arbitration, or negotiation protocols between agents.

**Q: What is agent robustness?**

A: The ability of an agent system to handle errors, unexpected inputs, and failures gracefully without complete breakdown.

**Q: What monitoring metrics are important for agents?**

A: Task completion rate, response time, error frequency, resource usage, decision quality, and inter-agent communication efficiency.

## Section 14: Workflow Tools

**Q: What is n8n?**

A: A workflow automation tool with visual interface for connecting apps, APIs, and AI models without extensive coding.

**Q: What are n8n nodes?**

A: Individual components in n8n that perform specific actions (API calls, data transformation, conditional logic, AI operations).

**Q: How can n8n integrate with AI agents?**

A: Through HTTP requests to agent APIs, webhook triggers, direct LLM integrations, and custom function nodes.

**Q: What is the benefit of visual workflow tools?**

A: Faster development, easier debugging, clear logic visualization, reusable components, and lower maintenance overhead.

**Q: When should you use workflow tools vs code?**

A: Workflow tools for integration-heavy tasks and rapid prototyping; code for complex algorithms, performance-critical systems, and custom logic.

## Section 15: API Development

**Q: What is Flask?**

A: A lightweight Python web framework for building APIs and web applications with minimal boilerplate.

**Q: What is a RESTful API?**

A: An API design pattern using HTTP methods (GET, POST, PUT, DELETE) for resource manipulation with stateless communication.

**Q: What are common Flask decorators for routes?**

A: `@app.route()` for defining endpoints, with `methods` parameter for HTTP verbs (GET, POST, etc.).

**Q: What is Streamlit?**

A: A Python framework for creating interactive web applications and demos with minimal code, ideal for AI/ML prototypes.

**Q: How do you structure a Flask API for AI applications?**

A: Separate routes for prediction, health checks, model management; use blueprints for organization; implement proper error handling.

**Q: What are essential API endpoints for AI apps?**

A: `/predict` (inference), `/health` (status check), `/models` (model info), `/feedback` (user input), `/metrics` (monitoring).

**Q: How do you handle streaming responses in Flask?**

A: Use generators with `yield` and set `Content-Type: text/event-stream` for server-sent events (SSE).

## Section 16: MLOps Principles

**Q: What is MLOps?**

A: Practices combining machine learning, DevOps, and data engineering to deploy and maintain ML models in production reliably.

**Q: What are the key MLOps components?**

A: Model versioning, experiment tracking, automated testing, CI/CD pipelines, monitoring, and model registry.

**Q: What is model versioning?**

A: Tracking different iterations of ML models with metadata, enabling reproducibility and rollback capabilities.

**Q: What is continuous integration for ML?**

A: Automated testing of code, data validation, and model quality checks triggered by code commits.

**Q: What is continuous deployment for ML?**

A: Automated pipeline to deploy validated models to production environments with minimal manual intervention.

**Q: What should be monitored in production ML systems?**

A: Model performance metrics, data drift, prediction latency, resource usage, error rates, and data quality.

**Q: What is model drift?**

A: Degradation in model performance over time due to changes in data distribution or target variable relationships.

## Section 17: Cloud Platforms

**Q: What is Google Vertex AI?**

A: Google Cloud's unified ML platform for building, deploying, and scaling AI models with integrated tools and services.

**Q: What is Azure ML Studio?**

A: Microsoft's cloud platform for building, training, and deploying ML models with visual designer and code-based options.

**Q: What are key features of Vertex AI?**

A: AutoML, custom training, model deployment, feature store, pipelines, and model monitoring.

**Q: What are key features of Azure ML?**

A: Designer (visual interface), automated ML, compute management, model registry, and deployment endpoints.

**Q: How do you deploy models on Vertex AI?**

A: Create model resource, upload model artifacts, create endpoint, deploy model to endpoint, configure traffic split.

**Q: How do you deploy models on Azure ML?**

A: Register model, create inference configuration, define deployment configuration, deploy to endpoint (ACI or AKS).

**Q: What is model serving?**

A: Hosting trained models as APIs that receive requests, perform inference, and return predictions in real-time or batch mode.

## Section 18: Database Integration

**Q: What is Cosmos DB?**

A: Microsoft's globally distributed NoSQL database service with multiple API support (SQL, MongoDB, Cassandra, Gremlin, Table).

**Q: What is Azure SQL?**

A: Microsoft's cloud relational database service based on SQL Server, offering PaaS with automated management.

**Q: How do AI applications use databases?**

A: Store user data, conversation history, embeddings for RAG, model predictions, feedback, and application state.

**Q: What is a vector database?**

A: A specialized database optimized for storing and querying high-dimensional vectors (embeddings) with similarity search.

**Q: What are common vector databases?**

A: Pinecone, Weaviate, Qdrant, ChromaDB, and Azure Cosmos DB for MongoDB vCore (with vector search).

**Q: How do you connect Flask to databases?**

A: Use libraries like pymongo (MongoDB), pyodbc (SQL), psycopg2 (PostgreSQL), or ORMs like SQLAlchemy.

## Section 19: RAG Architecture

**Q: What is RAG (Retrieval Augmented Generation)?**

A: A technique combining information retrieval with LLM generation to provide accurate, up-to-date responses grounded in external knowledge.

**Q: What are the components of RAG?**

A: Document ingestion, chunking, embedding generation, vector storage, retrieval, prompt augmentation, and generation.

**Q: What are embeddings?**

A: Numerical vector representations of text that capture semantic meaning and relationships, allowing mathematical comparison of textual similarity in high-dimensional space.

**Q: What are embedding models?**

A: Neural networks that convert text into dense vector representations capturing semantic meaning for similarity search.

**Q: Name some popular embedding models.**

A: OpenAI text-embedding-ada-002, OpenAI text-embedding-3-small, OpenAI text-embedding-3-large, Cohere embed-v3, Google PaLM embeddings, Sentence-BERT (SBERT), all-MiniLM-L6-v2, and Azure OpenAI embedding models.

**Q: What is document chunking in RAG?**

A: Breaking large documents into smaller segments to fit within LLM context windows and improve retrieval precision.

**Q: What is Azure AI Search?**

A: Microsoft's cloud search service with vector search capabilities, supporting hybrid retrieval (keyword + semantic).

**Q: What is semantic search?**

A: Retrieval based on meaning and context rather than exact keyword matches, using vector similarity.

**Q: What is hybrid search in RAG?**

A: Combining keyword-based (BM25) and semantic (vector) search to leverage strengths of both approaches.

## **Section 20: Ethical AI**

**Q: What are the key principles of ethical AI?**

A: Fairness, accountability, transparency, privacy, security, human oversight, and societal benefit.

**Q: What is AI bias?**

A: Systematic errors in AI predictions that unfairly favor or discriminate against certain groups due to training data or design.

**Q: How can you detect bias in AI models?**

A: Analyze performance across demographic groups, test with diverse inputs, use fairness metrics, and conduct adversarial testing.

**Q: What is explainable AI (XAI)?**

A: Techniques making AI decisions interpretable and understandable to humans, showing reasoning behind predictions.

**Q: What is data privacy in AI context?**

A: Protecting personal information used for training or inference, ensuring compliance with regulations (GDPR, CCPA).

**Q: What is responsible AI development?**

A: Building AI systems considering ethical implications, potential harms, societal impact, and implementing safeguards.

**Q: How do you ensure AI accountability?**

A: Document decisions, maintain audit trails, establish clear ownership, implement review processes, and enable human oversight.